ENSA BM

Architecture Logicielle

Rapport de Travaux Pratiques

TP2

Du Monolithe Simple à l'Architecture Distribuée

Encadrant : Pr. ELBAGHAZAOUI

Année universitaire : 2025 / 2026

```
psql (17.6)
WARNING: Console code page (437) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
Type "help" for help.

ecommerce=# \dt
            List of relations
 Schema |    Name    | Type  |  Owner
--------+------------+-------+----------
 public | categories | table | postgres
 public | customers  | table | postgres
 public | orders     | table | postgres
 public | products   | table | postgres
(4 rows)


ecommerce=#
```

TP2 — Monolithe Modulaire

2.1 Objectifs

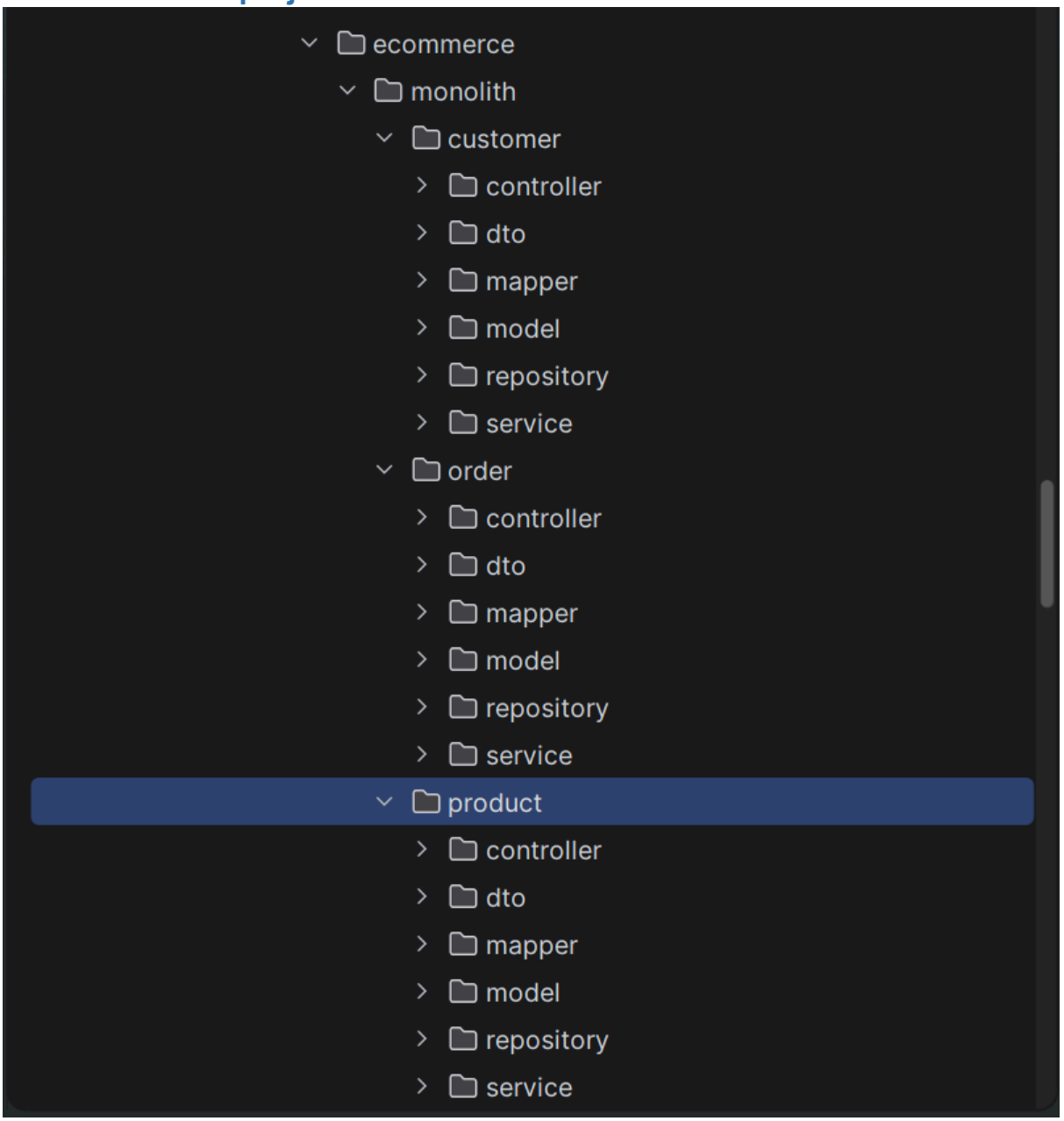
- Restructurer le monolithe en modules métier distincts
- Définir des boundaries claires entre domaines
- Implémenter des interfaces de communication entre modules
- Appliquer les design patterns : DTO, Mapper, Service Layer
- Préparer le terrain pour une future migration microservices

2.2 Architecture modulaire

Le TP2 introduit une séparation claire entre 3 modules indépendants :

Module	Rôle	Dépendances
Product	Gestion des produits	Aucune
Customer	Gestion des clients	Aucune
Order	Gestion des commandes	ProductService + CustomerService (jamais les Repositories)

2.3 Structure du projet



2.4 Pattern DTO + Mapper

Chaque module utilise des DTOs pour séparer la couche de présentation de la couche de persistance, et MapStruct pour le mapping automatique.

Category.java DemoApplication.java Customer.java ProductMapper.java

```java
package com.ecommerce.monolith.product.mapper;

import com.ecommerce.monolith.product.dto.ProductDto;
import com.ecommerce.monolith.product.dto.ProductRequest;
import com.ecommerce.monolith.product.model.Product;
import java.util.List;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;

@Mapper(componentModel = "spring")  1 usage  laarichothman-prog
public interface ProductMapper {
    ProductDto toDTO(Product product);  no usages  laarichothman-prog

    List<ProductDto> toDTOList(List<Product> products);  no usages  laarichothman-prog

    @Mapping(target = "id", ignore = true)  no usages  laarichothman-prog
    @Mapping(target = "category", ignore = true)
    Product toEntity(ProductRequest request);

    @Mapping(target = "id", ignore = true)  no usages  laarichothman-prog
    @Mapping(target = "category", ignore = true)
    void updateEntity(ProductRequest request, @MappingTarget Product product);
}
```

Category.java DemoApplication.java ProductServiceImpl.java

```java
public class ProductServiceImpl implements ProductService {
    private final ProductRepository repository;  7 usages
    private final ProductMapper mapper;  6 usages
    @Override  no usages  laarichothman-prog
    @Transactional(readOnly = true)
    public List<ProductDto> getAllProducts() { return mapper.toDTOList(repository.findAll()); }

    @Override  no usages  laarichothman-prog
    @Transactional(readOnly = true)
    public ProductDto getProductById(Long id) {
        Product product = repository
                .findById(id)
                .orElseThrow(() -> new IllegalArgumentException("Product not found with id: " + id));
        return mapper.toDTO(product);
    }
    @Override  no usages  laarichothman-prog
    public ProductDto createProduct(ProductRequest request) {
        Product product = mapper.toEntity(request);
        return mapper.toDTO(repository.save(product));
    }
    @Override  no usages  laarichothman-prog
    public ProductDto updateProduct(Long id, ProductRequest request) {
        Product product = repository
                .findById(id)
                .orElseThrow(() -> new IllegalArgumentException("Product not found with id: " + id));
        mapper.updateEntity(request, product);
        return mapper.toDTO(repository.save(product));
    }
    @Override  no usages  laarichothman-prog
    public void deleteProduct(Long id) {
        if (!repository.existsById(id)) {
            throw new IllegalArgumentException("Product not found with id: " + id);
        }
        repository.deleteById(id);
    }
```

2.5 Module Customer

2.5.1 Entité Customer

```java
Category.java    DemoApplication.java    Customer.java

@Entity  3 usages  laarichothman-prog
@Table(name = "customers")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Customer {
    @Id  no usages
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "First name is required")  no usages
    @Column(nullable = false)
    private String firstName;

    @NotBlank(message = "Last name is required")  no usages
    @Column(nullable = false)
    private String lastName;

    @NotBlank(message = "Email is required")  no usages
    @Email(message = "Email must be valid")
    @Column(nullable = false, unique = true)
    private String email;

    private String phone;  no usages
}
```

2.5.2 Méthode customerExists()

Le TP2 demandait spécifiquement d'ajouter une méthode de vérification d'existence du client. Cette méthode est utilisée par OrderServiceImpl avant chaque création de commande.

2.6 Module Order

2.6.1 Dépendances inter-modules

Champ	Type	Contrainte
id	Long	@GeneratedValue IDENTITY
firstName	String	@NotBlank
lastName	String	@NotBlank
email	String	@Email, unique=true
phone	String	Optionnel

```java
package com.ecommerce.monolith.customer.service;

import ...

public interface CustomerService {
    List<CustomerDTO> getAllCustomers();
    CustomerDTO getCustomerById(Long id);
    CustomerDTO createCustomer(CustomerRequest request);
    CustomerDTO updateCustomer(Long id, CustomerRequest request);
    void deleteCustomer(Long id);
    boolean customerExists(Long id);
}
```

OrderServiceImpl injecte CustomerService et ProductService, jamais leurs repositories. C'est la règle fondamentale du monolithe modulaire.

```java
package com.ecommerce.monolith.order.service;

import ...

public interface OrderService {
    List<OrderDTO> getAllOrders();
    OrderDTO getOrderById(Long id);
    OrderDTO createOrder(OrderRequest request);
    void cancelOrder(Long id);
    List<OrderDTO> getOrdersByCustomer(Long customerId);
}
```

2.6.2 Endpoint historique des commandes

Le TP2 demandait de créer un endpoint pour obtenir l'historique des commandes d'un client :

```
GET /api/orders/customer/{customerId}
```

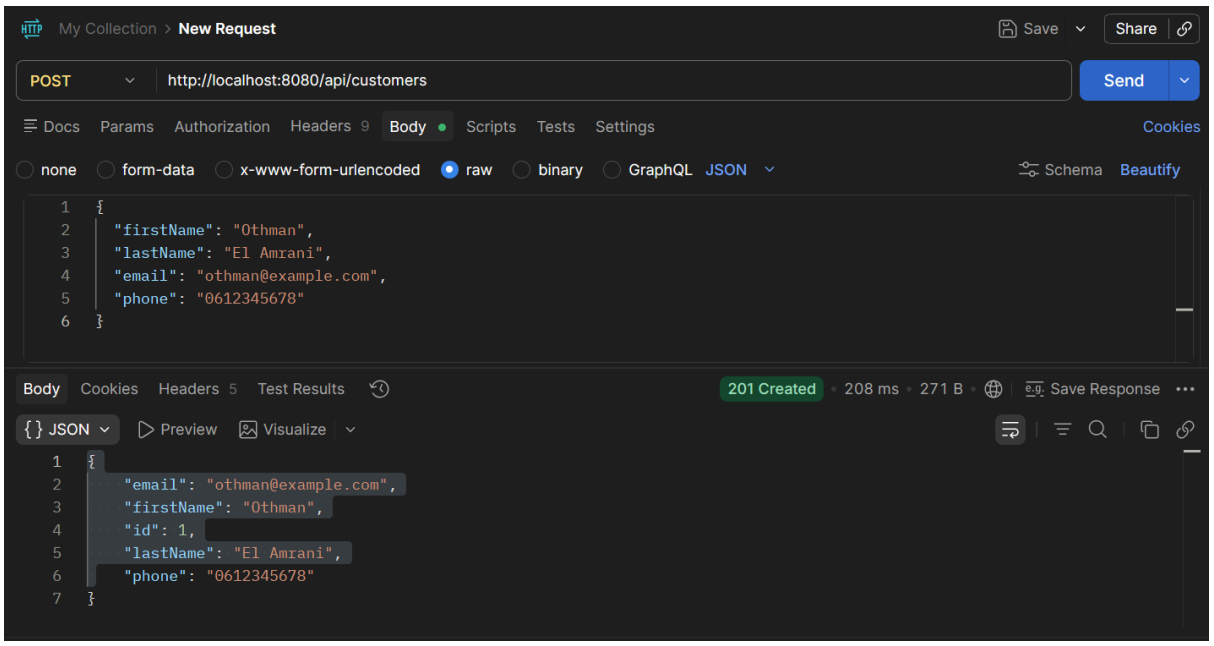
📸 **SCREENSHOT**

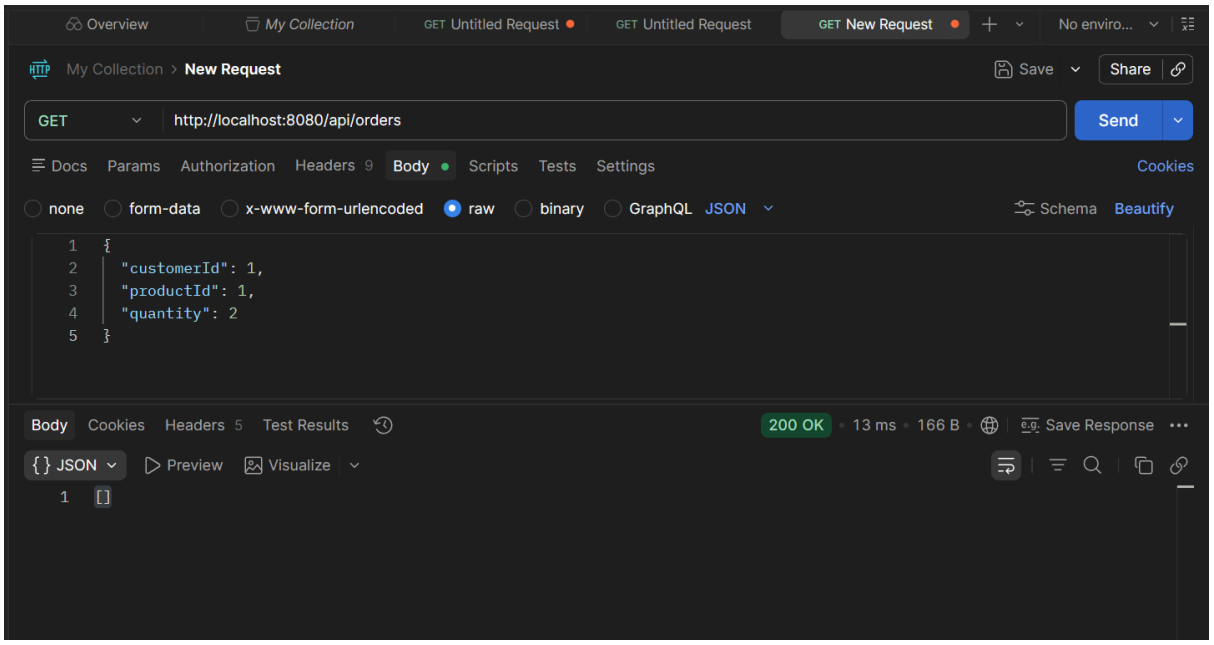
GET /api/orders/customer/{id} — Historique commandes

Capture Postman montrant la requête GET /api/orders/customer/1 et la réponse avec la liste des commandes du client

2.7 Tests Postman complets

Méthode	URL	Description	Status
POST	/api/customers	Créer un client	201
GET	/api/customers	Liste des clients	200
POST	/api/products	Créer un produit	201
POST	/api/orders	Créer une commande	201
GET	/api/orders/customer/{id}	Historique commandes	200
PATCH	/api/orders/{id}/cancel	Annuler une commande	204





TP3 — Architecture Distribuée

3.1 Objectifs